VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

REPORT

FINAL PROJECT

Xv6 - Unix Utilities and System Call

CSC10007 - Operating System

Team members:

24127457 - Nguyen Ho Nam
24127561 - Dang The Tony

Instructors:

Theory: Dr. Vu Thi My Hang
Lab: Le Quoc Hoa

Ho Chi Minh City, March 19, 2026

Contents

1 Introduction **2**

2 User Programs **4**

2.1 xargs 4

2.1.1 Description 4

2.1.2 Approach 4

2.1.3 Challenges 4

2.2 tree 5

2.2.1 Description 5

2.2.2 Approach 5

2.2.3 Challenges 5

3 System Calls **7**

3.1 Overview: System Call Mechanism in xv6 7

3.2 trace 7

3.2.1 Description 7

3.2.2 Approach 8

3.2.3 Challenges 8

3.3 procinfo 9

3.3.1 Description 9

3.3.2 Data Structure 9

3.3.3 Approach 9

3.3.4 Challenges 10

4 Conclusion **11**

Work Distribution **12**

References **13**

Chapter 1

Introduction

This project is based on the **xv6** operating system — a simple, Unix-like teaching OS developed at MIT — running on the RISC-V architecture. The project focuses on both *user-level utilities* and *kernel-level system calls*, giving us hands-on experience with the complete path from user programs down into the operating system kernel.

The project consists of two main parts:

- **User Programs:** `xargs` and `tree`
- **System Calls:** `trace` and `procinfo`

All implementations are developed and tested in the xv6 environment on Linux using QEMU as the hardware emulator.

The table below summarizes all files that were modified or created during this project:

File	Purpose
`user/xargs.c`	xargs user program
`user/tree.c`	tree user program
`user/trace.c`	trace user program (provided by lab)
`user/procinfo.c`	procinfo test program
`kernel/proc.h`	Added `tracemask` field to `struct proc`
`kernel/proc.c`	Modified `fork()` to propagate trace mask to child
`kernel/procinfo.h`	New header defining `struct procinfo`
`kernel/syscall.h`	Added `SYS_trace` (22) and `SYS_procinfo` (23)
`kernel/syscall.c`	Extended dispatch table; added name table; modified `syscall()`
`kernel/sysproc.c`	Implemented `sys_trace()` and `sys_procinfo()`
`user/user.h`	Added user-space function prototypes
`user/usys.pl`	Added `entry()` stubs for new system calls
`Makefile`	Registered new programs in `UPROGS`

Chapter 2

User Programs

2.1 xargs

2.1.1 Description

The **xargs** program reads lines from standard input and executes a given command by appending each input line as additional arguments. This is a simplified version of the UNIX `xargs` utility (without the multi-argument optimization).

2.1.2 Approach

1. **Initialize argument array:** All arguments already supplied on the command line (`argv[1..argc-1]`) are copied into a local `args[]` array upfront. These form the fixed prefix of every command invocation.

2. **Read input character by character:** Standard input (file descriptor 0) is read one byte at a time into a buffer until a newline character (`'\n'`) is encountered.

3. **Execute command per line:** When a newline is found, the buffer is null-terminated and appended as the next argument in `args[]`. A child process is created with **fork()**, which then calls **exec()** to run the command. The parent waits for the child to complete using **wait()** before processing the next input line.

4. **Repeat until EOF:** When `read()` returns 0, the loop exits cleanly.

2.1.3 Challenges

- **Argument boundary:** The combined argument count must never exceed `MAXARG` (defined in `kernel/param.h`). One extra slot is reserved for the stdin line and one more for the null sentinel.

- **Buffer reuse:** The same buffer is reused across iterations. Resetting the write index to 0 after each `wait()` is sufficient because `exec()` in the child replaces the entire address space, leaving the parent's buffer valid for the next round.

- **Partial last line:** If the last line of input has no trailing newline, the content in the buffer is never flushed. This is acceptable and consistent with standard `xargs` behavior.

2.2 tree

2.2.1 Description

The **tree** program displays the file system hierarchy starting from a given directory in a tree-like visual format. It accepts one optional argument; if omitted, it defaults to the current directory (`"."`).

2.2.2 Approach

The program uses a recursive helper function `tree(char *path, int depth)` which processes one path at each call level.

1. **Open and stat the path:** The target path is opened with `open()` and its metadata is retrieved with `fstat()`. If the path is not a directory (`stat.type != T_DIR`), it is printed as a plain file entry with depth-based indentation and the recursion stops at that node.

2. **Print the directory name:** If the entry is a directory, it is printed with a trailing `/` character to visually distinguish it from regular files.

3. **Apply indentation:** Two spaces are printed per depth level, so nested entries are visually offset from their parent.

4. **Iterate over directory entries:** The directory is read entry by entry using `read()` with `struct dirent`. Entries with `inum == 0` (unused slots) are skipped. Entries named `"."` and `".."` are also skipped to prevent infinite recursion.

5. **Build child path and recurse:** For each valid entry, the child path is constructed by concatenating the parent path, a slash, and the entry name. The function then calls itself recursively with `depth + 1`.

2.2.3 Challenges

- **Infinite recursion prevention:** Skipping `"."` and `".."` entries is critical. Without this guard, the traversal would loop indefinitely by re-entering the same or parent directory.

- **Path construction:** The entry name stored in `struct dirent` is a fixed-length field (`DIRSIZ = 14` bytes) and is not necessarily null-terminated. `memmove` is used instead of `strcpy` and the byte at position `DIRSIZ` is explicitly set to `0` to ensure a valid C string.

- **Error handling:** Both `open()` and `fstat()` failures are handled gracefully by printing an error message to stderr and returning early, so the rest of the tree is still displayed.

Chapter 3

System Calls

3.1 Overview: System Call Mechanism in xv6

Before presenting the individual implementations, it is helpful to understand the system call pipeline in xv6, since both `trace` and `procinfo` follow the same path:

1. A user program calls a C wrapper (e.g., `trace(mask)`).
2. The wrapper is an assembly stub generated from `user/usys.pl` that issues a RISC-V `ecall` instruction, placing the syscall number in register `a7`.
3. The hardware traps into kernel mode and jumps to `usertrap()` in `kernel/trap.c`.
4. `usertrap()` calls `syscall()` in `kernel/syscall.c`.
5. `syscall()` reads the syscall number from `a7`, dispatches to the corresponding `sys_xxx()` function, and stores the return value in register `a0`.
6. Control returns to user space via `usertrapret()`.

Adding a new system call therefore requires coordinated changes across multiple files: the syscall number table, the dispatch table, the kernel implementation, and the user-space stubs.

3.2 trace

3.2.1 Description

The **trace** system call enables per-process system call tracing. It takes a single integer *mask* argument: each bit position in the mask corresponds to a syscall number. When a traced syscall returns, the kernel prints the process ID, the syscall name, and the return value. The trace mask is inherited by child processes created with `fork()`.

3.2.2 Approach

1. **Extend `struct proc` (`kernel/proc.h`):** A new integer field `tracemask` is added to the process descriptor. It defaults to 0 (no tracing) for all newly allocated processes since `allocproc()` already zeroes the structure.

2. **Register the syscall number (`kernel/syscall.h`):** The constant `SYS_trace` is defined as 22, following the existing numbering scheme.

3. **Implement `sys_trace()` (`kernel/sysproc.c`):** The function retrieves the mask argument from user registers using `argint()` and stores it into `myproc()->tracemask`. It returns 0 on success.

4. **Propagate the mask in `fork()` (`kernel/proc.c`):** After the child process is created, its `tracemask` is set equal to the parent's. This single assignment ensures that any descendant process is automatically traced, which is required for commands like `trace 2 usertests forkforkfork`.

5. **Print trace output in `syscall()` (`kernel/syscall.c`):** After dispatching the syscall and storing the return value in `a0`, the function checks whether the bit corresponding to the syscall number is set in `p->tracemask` using the expression `(p->tracemask >> num) & 1`. If set, it prints the PID, syscall name (looked up from a `char *syscallnames[]` array), and return value.

6. **Add user-space stubs (`user/user.h`, `user/usys.pl`):** A function prototype `int trace(int)` is added to `user.h`, and `entry("trace")` is added to `usys.pl` so the Perl script generates the corresponding `ecall` assembly stub.

3.2.3 Challenges

- **Understanding the syscall dispatch path:** We had to carefully trace the execution flow from `ecall` through `usertrap` and `syscall()` to identify the correct insertion point for tracing logic — specifically, *after* the dispatch returns but *before* returning to user space.

- **Mapping numbers to names:** xv6's dispatch table uses a function pointer array indexed by syscall number. We added a parallel `char *syscallnames[]` array using C's designated initializer syntax (`[SYS_trace] = "trace"`) to keep the mapping consistent and easy to extend.

- **Fork inheritance:** Without modifying `fork()`, child processes would start with `tracemask = 0` and produce no output even if their parent is traced. One line of code in `fork()` is all that is needed to fix this.

3.3 procinfo

3.3.1 Description

The **procinfo** system call retrieves runtime information about any process given its PID. It takes two arguments: an integer `pid` and a pointer to a `struct procinfo` in user space. On success, it fills the struct with the target process's PID, parent PID, state, memory size (bytes), and name, then returns 0. It returns -1 if the process is not found or if copying to user space fails.

3.3.2 Data Structure

A new header file `kernel/procinfo.h` was created to define the struct, shared between kernel and user space:

Field	Type	Description
`pid`	`int`	Process ID
`ppid`	`int`	Parent process ID (0 if no parent)
`state`	`int`	Process state (maps to `enum procstate` in `proc.h`)
`sz`	`uint64`	Size of process memory in bytes
`name`	`char[16]`	Process name

The `state` field maps to the values: `UNUSED=0`, `USED=1`, `SLEEPING=2`, `RUNNABLE=3`, `RUNNING=4`, `ZOMBIE=5`.

3.3.3 Approach

1. **Create `struct procinfo` header (`kernel/procinfo.h`):** The struct is defined in its own header to allow both kernel code and user programs to include it cleanly without circular dependencies.

2. **Register the syscall number (`kernel/syscall.h`):** The constant `SYS_procinfo` is defined as 23.

3. **Implement `sys_procinfo()` (`kernel/sysproc.c`):**

   - Extract the two arguments: `pid` using `argint()` and the user-space destination address using `argaddr()`.
   - Perform a linear scan over the global process table `proc[NPROC]` to find the process matching the given `pid`.

- Once found, populate a local `struct procinfo` in kernel space from the corresponding `struct proc`: PID, parent PID (with a null-parent guard for PID 1), state, memory size, and name (copied with `safestrcpy`).
- Transfer the kernel-side struct to user space using `copyout()`, which translates the user virtual address through the process page table and writes safely.
- Return 0 on success, -1 if the PID is not found or if `copyout()` fails.

4. **Add user-space stubs (`user/user.h`, `user/usys.pl`):** A forward declaration `struct procinfo` and the prototype `int procinfo(int, struct procinfo *)` are added to `user.h`. `entry("procinfo")` is added to `usys.pl`.

5. **Write a test program (`user/procinfo.c`):** A simple program calls `procinfo(1, &info)` to query the `init` process and prints all returned fields to verify correctness.

3.3.4 Challenges

- **Kernel-to-user data transfer:** The kernel cannot dereference a user-space pointer directly because the kernel and each user process have separate virtual address spaces. Writing to the user pointer without going through `copyout()` would either cause a page fault or silently corrupt memory. Using `copyout(pagetable, dstvirt, src, len)` is the correct and safe approach, as consistently used elsewhere in the xv6 kernel (e.g., in `sys_read`).

- **Null parent for `init`:** The `init` process (PID 1) has no parent, so `p->parent` is `NULL`. A conditional expression checks for this case before accessing `p->parent->pid`, preventing a null pointer dereference.

- **Accessing the process table:** The global array `proc[NPROC]` is defined in `kernel/proc.c` but not exported in its header. Declaring it as `extern struct proc proc[NPROC]` in `sysproc.c` grants access without modifying the existing interface of `proc.c`.

- **Separate header for shared struct:** Defining `struct procinfo` in `kernel/procinfo.h` rather than inline inside `proc.h` or `sysproc.c` avoids include-order issues and keeps the struct accessible to both kernel implementation files and user programs.

Chapter 4

Conclusion

Through this project, we gained a much deeper understanding of several fundamental operating system concepts:

- **UNIX utilities:** Implementing `xargs` and `tree` demonstrated how classic UNIX tools are built entirely from simple OS primitives: `fork`, `exec`, `wait`, `open`, `read`, and `stat`.
- **System call mechanism:** Adding `trace` and `procinfo` required understanding every layer of the syscall path — from the user-space assembly stub through the RISC-V `ecall` trap, the kernel dispatcher, and back to user space.
- **Process management:** Working with `struct proc`, the global process table, and `fork()` gave us direct insight into how the kernel tracks and manages running processes.
- **Kernel-user boundary:** The requirement to use `copyout()` and the argument-extraction helpers `argint()`/`argaddr()` reinforced why the kernel must treat all user-supplied pointers with suspicion and always transfer data through validated, page-table-aware paths.

Work Distribution

Student ID	Name	Tasks	Contribution
24127561	Dang The Tony	`xargs`, `tree`	50%
24127457	Nguyen Ho Nam	`trace`, `procinfo`	50%

Dang The Tony (24127561) was responsible for the two user-space programs. He implemented `xargs` (`user/xargs.c`), handling argument parsing, the character-by-character input loop, and the `fork`–`exec`–`wait` cycle. He also implemented `tree` (`user/tree.c`), including the recursive directory traversal, depth-based indentation, path construction, and error handling. Both programs were registered in the `Makefile` under `UPROGS`.

Nguyen Ho Nam (24127457) was responsible for the two kernel-level system calls. He implemented `trace` by extending `struct proc` with a `tracemask` field, modifying `fork()` to propagate the mask to child processes, and hooking into `syscall()` to print trace output. He implemented `procinfo` by creating the shared `kernel/procinfo.h` header, writing `sys_procinfo()` with a process-table scan and safe kernel-to-user copy via `copyout()`, and providing a test program `user/procinfo.c`. He also handled all user-space stubs (`user.h`, `usys.pl`) and syscall registration (`syscall.h`, `syscall.c`).

Both members collaborated on testing, debugging, and writing this report.

This project provided concrete, hands-on experience in both user-space programming and kernel development, bridging the gap between theoretical OS concepts and real implementation inside the xv6 kernel.

References

[1] MIT PDOS. *xv6 Lab: Unix Utilities*. 6.1810 Operating System Engineering, 2024. `https://pdos.csail.mit.edu/6.1810/2024/labs/util.html`

[2] MIT PDOS. *xv6 Lab: System Calls*. 6.1810 Operating System Engineering, 2023. `https://pdos.csail.mit.edu/6.1810/2023/labs/syscall.html`

[3] MIT PDOS. *xv6: a simple, Unix-like teaching operating system*. `https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf`

[4] Samuel E. Ghazi. *File Descriptors and Open Files* [Video]. YouTube, 2022. `https://www.youtube.com/watch?v=oWuVGDese4k`